C - LINUX - SBRK-BASED ALLOCATOR

Hmm

Heap Memory Manager

Algorithm Flowcharts & Logic Reference

HmmAlloc - HmmFree - hmmCalloc - hmmReAlloc - hmmSize

Subject	Custom replacement for malloc / free / calloc / realloc
Language / OS	GNU C (gcc) - Linux, heap grown and shrunk with sbrk()
Source files	**HmmAlloc.c** allocation, free-list helpers, calloc / realloc / size **HmmFree.c** free and block coalescing - **Hmm.h** block metadata and inline helpers **heapWrappers.c** libc symbol interposition - **Makefile** build
Strategy	Best fit (size-ordered doubly linked free list), boundary-tag coalescing, 16-byte alignment, global mutex (heap_lock)
Revision	1.1 - 26 October 2026

Contents

Fig. 1	System overview and call graph	2
Fig. 2	Block layout, metadata bits, neighbour navigation	3
Fig. 3	HmmAlloc() flowchart	4
Fig. 4	HmmFree() flowchart	5
Fig. 5a-f	Helper functions: find_block, split_block, insert_in_order, remove_from_free_list, expand_heap, merge_free_blocks	6
Fig. 6a-c	hmmCalloc, hmmReAlloc, hmmSize and the libc wrappers	11
Fig. 7	Worked example: allocation and free sequence	13

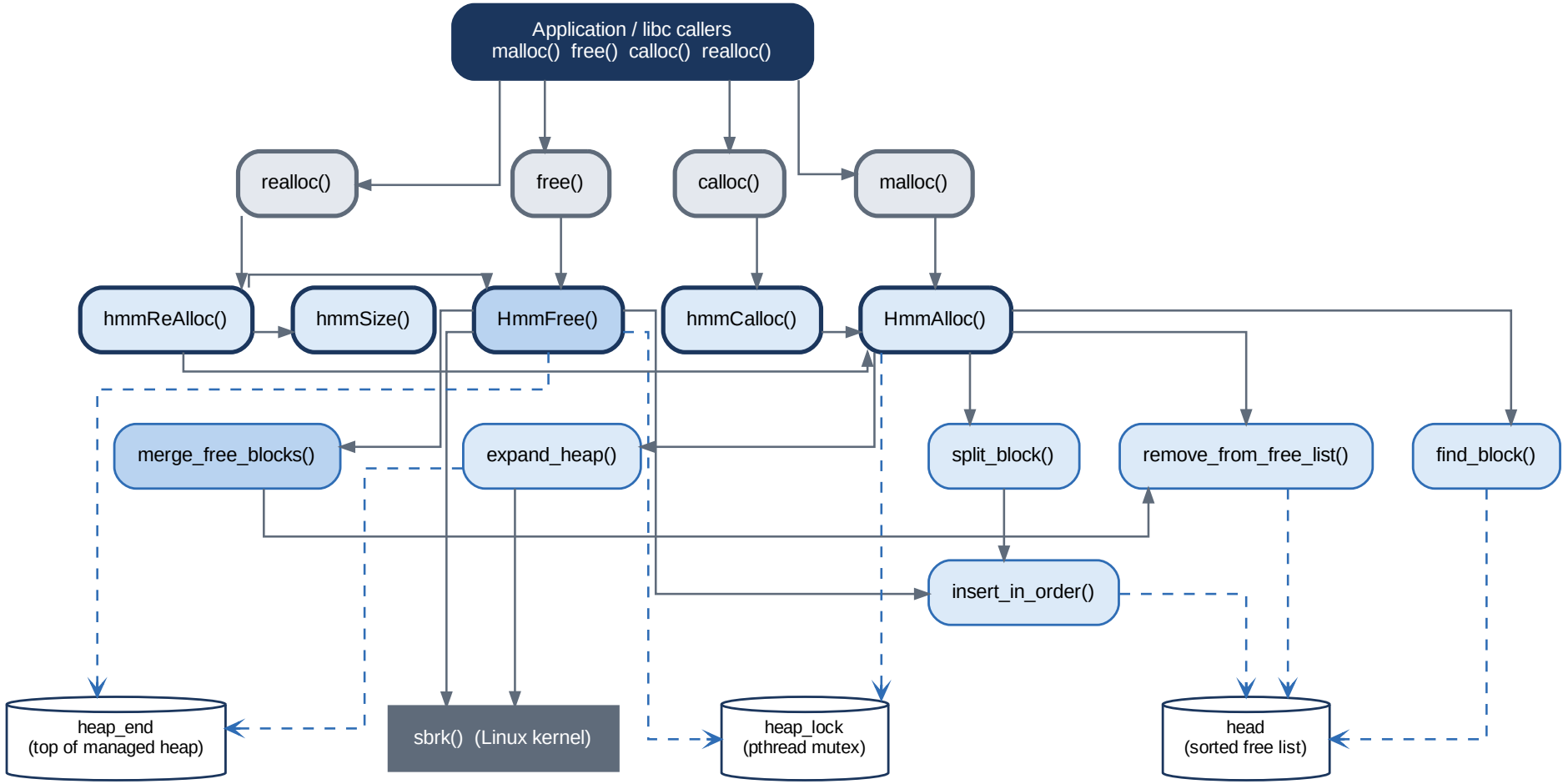
Flowchart legend



Fig. 1

System overview and call graph

Who calls whom: wrappers, public API, internal helpers, global state and the kernel



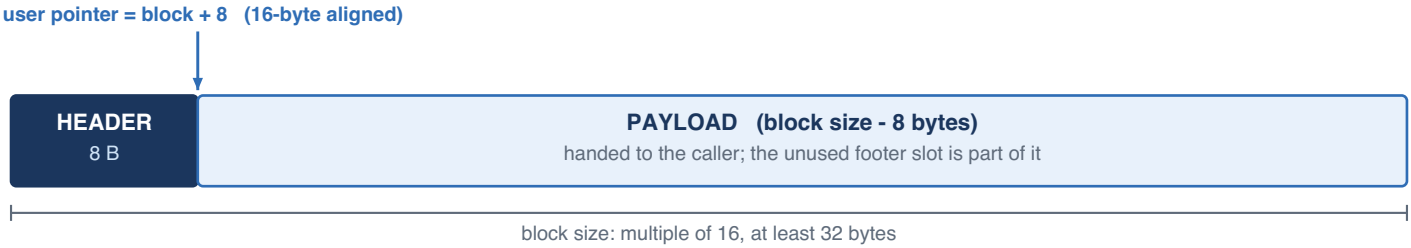
heapWrappers.c HmmAlloc.c HmmFree.c Bold outline = public function. Solid arrow = call. Dashed arrow = reads / writes global state.

Fig. 2

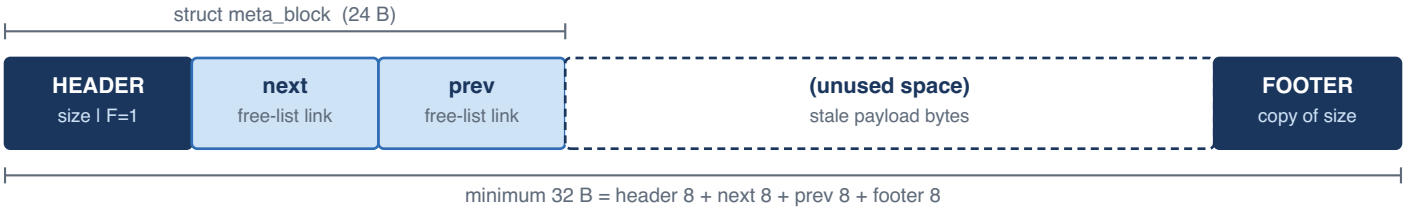
Block layout and metadata

Boundary-tag design: 8-byte header, optional 8-byte footer (free blocks only)

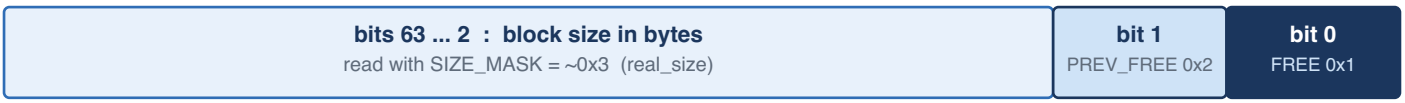
A. Allocated block



B. Free block

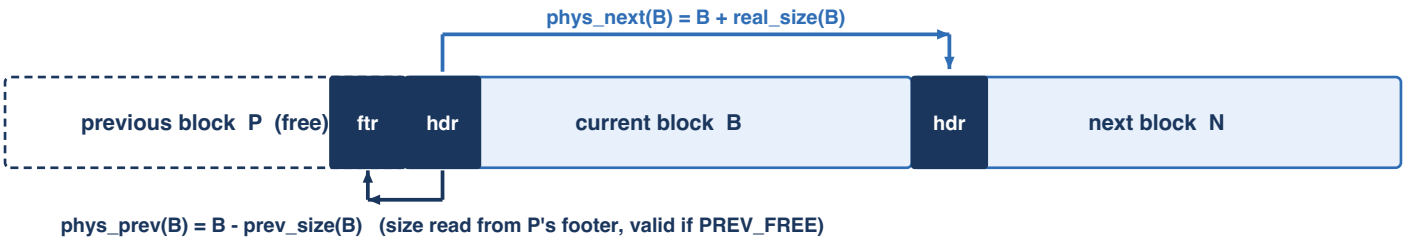


C. Header word (64 bit)



FREE = this block is on the free list. **PREV_FREE** = the block physically before this one is free, so its footer (at `block - 8`) holds a valid size.

D. Walking between physical neighbours



E. Key constants (Hmm.h)

Constant	Value	Role
ALIGN(n)	round up to 16	every block size is a multiple of 16; payload (block + 8) is 16-byte aligned
Minimum block	32 bytes	room for header, next, prev and footer once the block is freed
Split threshold	remainder >= 32 B	a larger-than-needed block is cut and the tail goes back to the free list
MIN_INCREMENT	128 KiB	smallest amount the heap grows by in expand_heap()
MIN_DECREMENT	128 KiB	a freed top block this big (or bigger) is returned to the OS

Fig. 3

HmmAlloc(size)

Best fit from the sorted free list, else grow the heap by 128 KiB or more. Runs under heap_lock

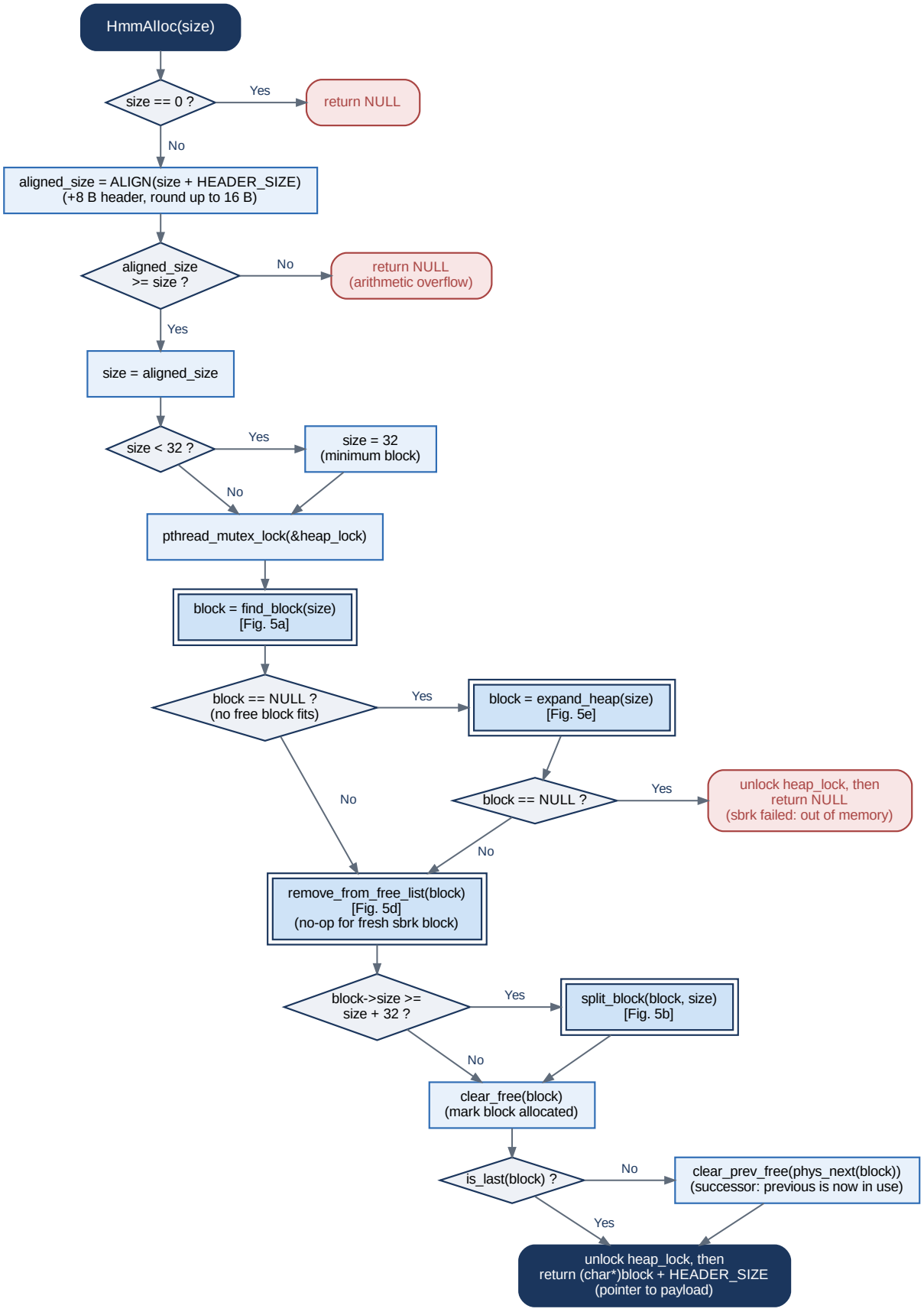


Fig. 4

HmmFree(ptr)

Coalesce with free neighbours, write footer and flags, then shrink the heap or file the block. Runs under heap_lock

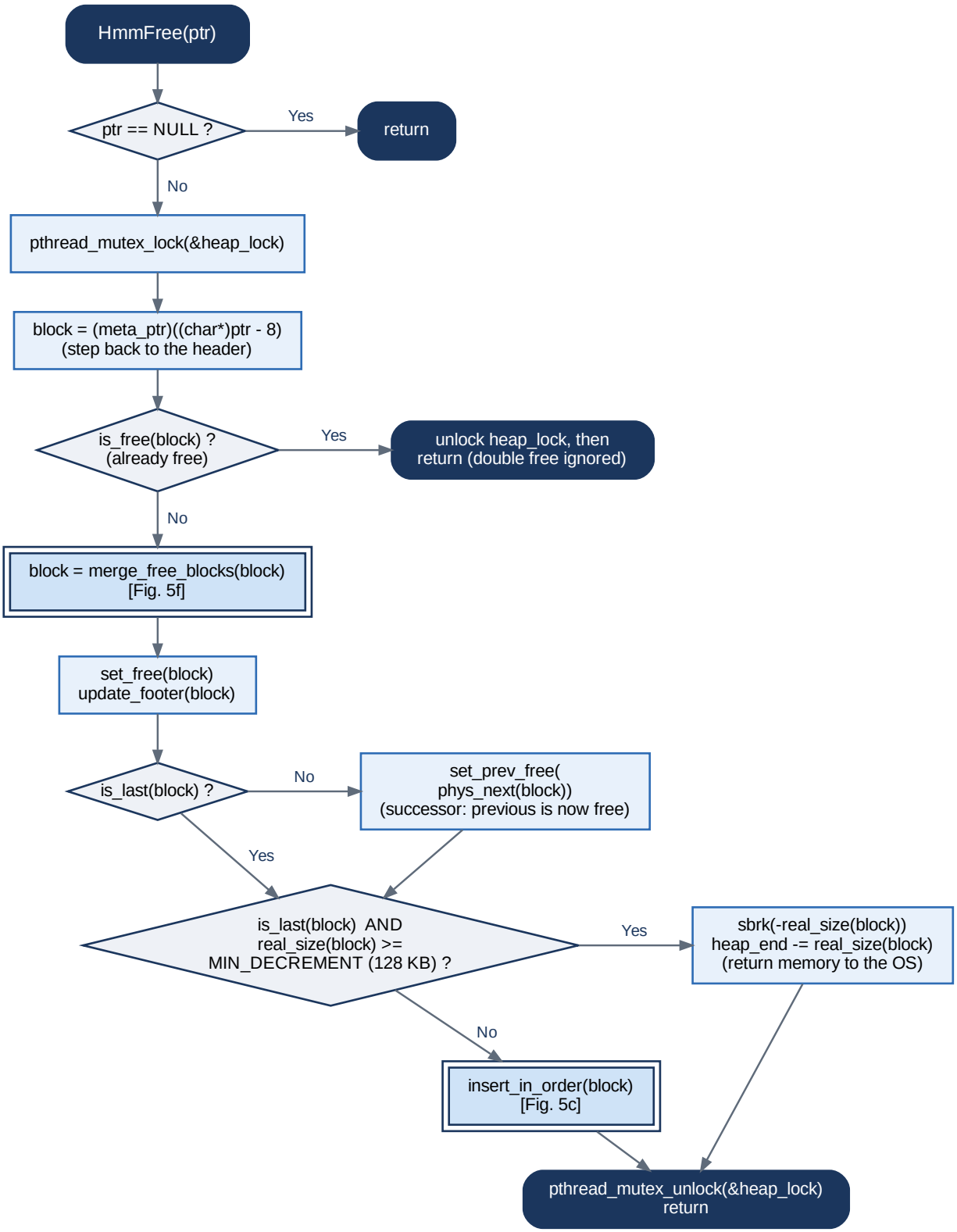


Fig. 5a

find_block(size)

Walks the size-sorted list; the first block that is large enough is the best fit

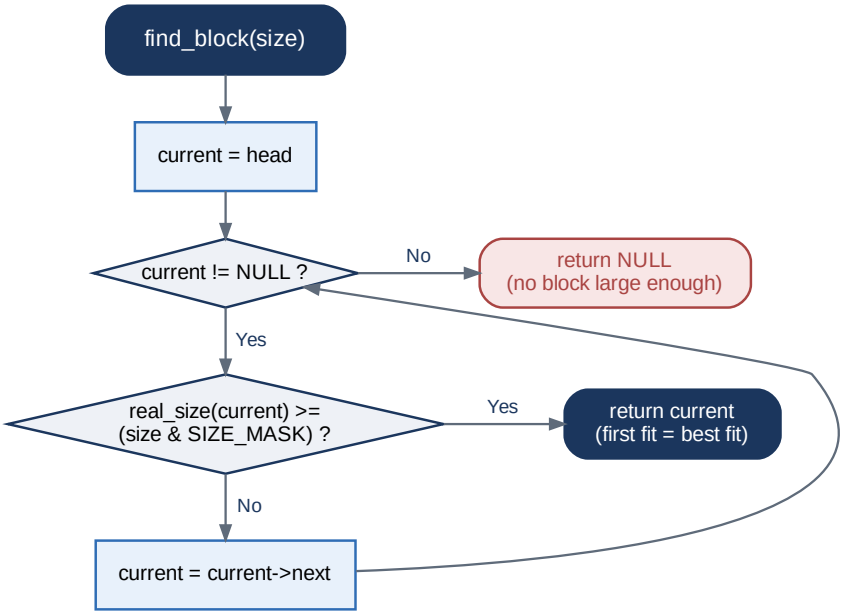


Fig. 5b

split_block(block, size)

Cuts a too-large block in two and returns the remainder to the free list

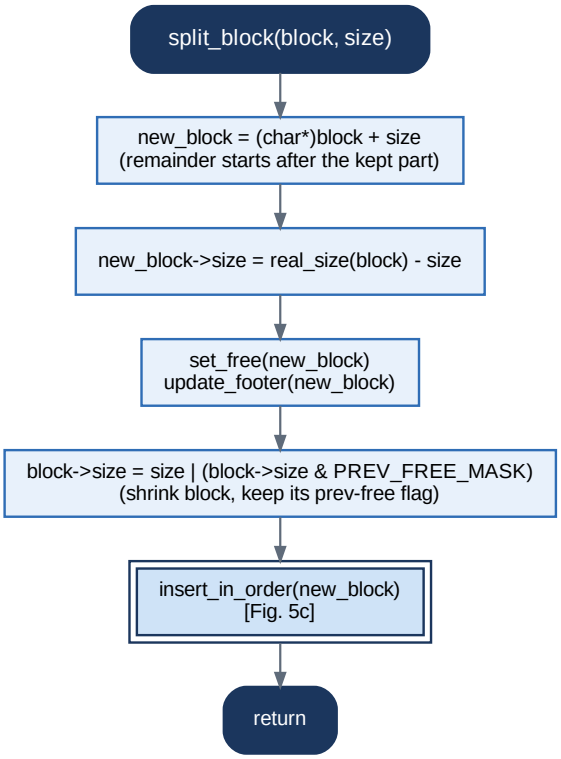


Fig. 5c

insert_in_order(block)

Keeps the doubly linked free list sorted by ascending size (an equal-sized block goes in front of its equals)

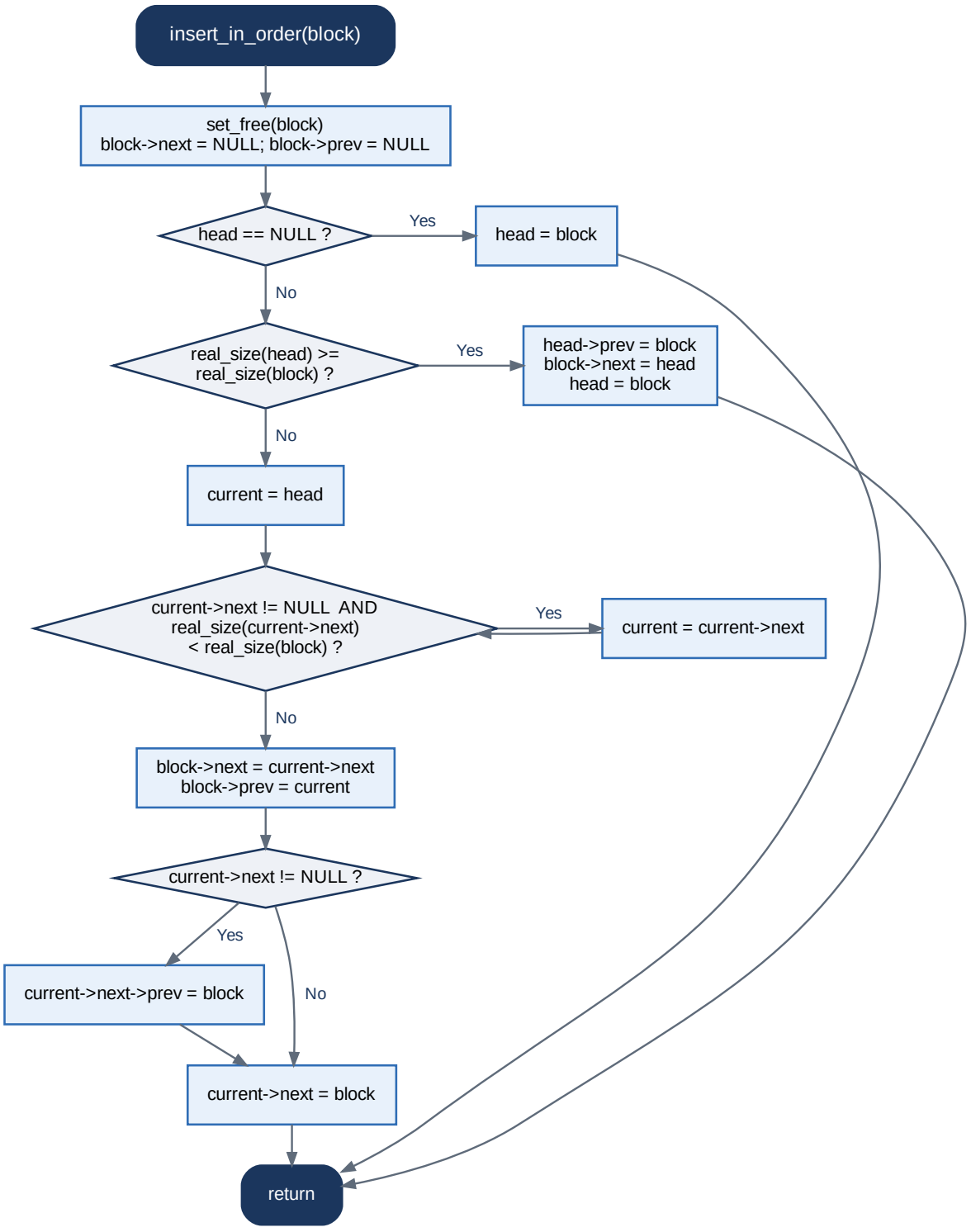
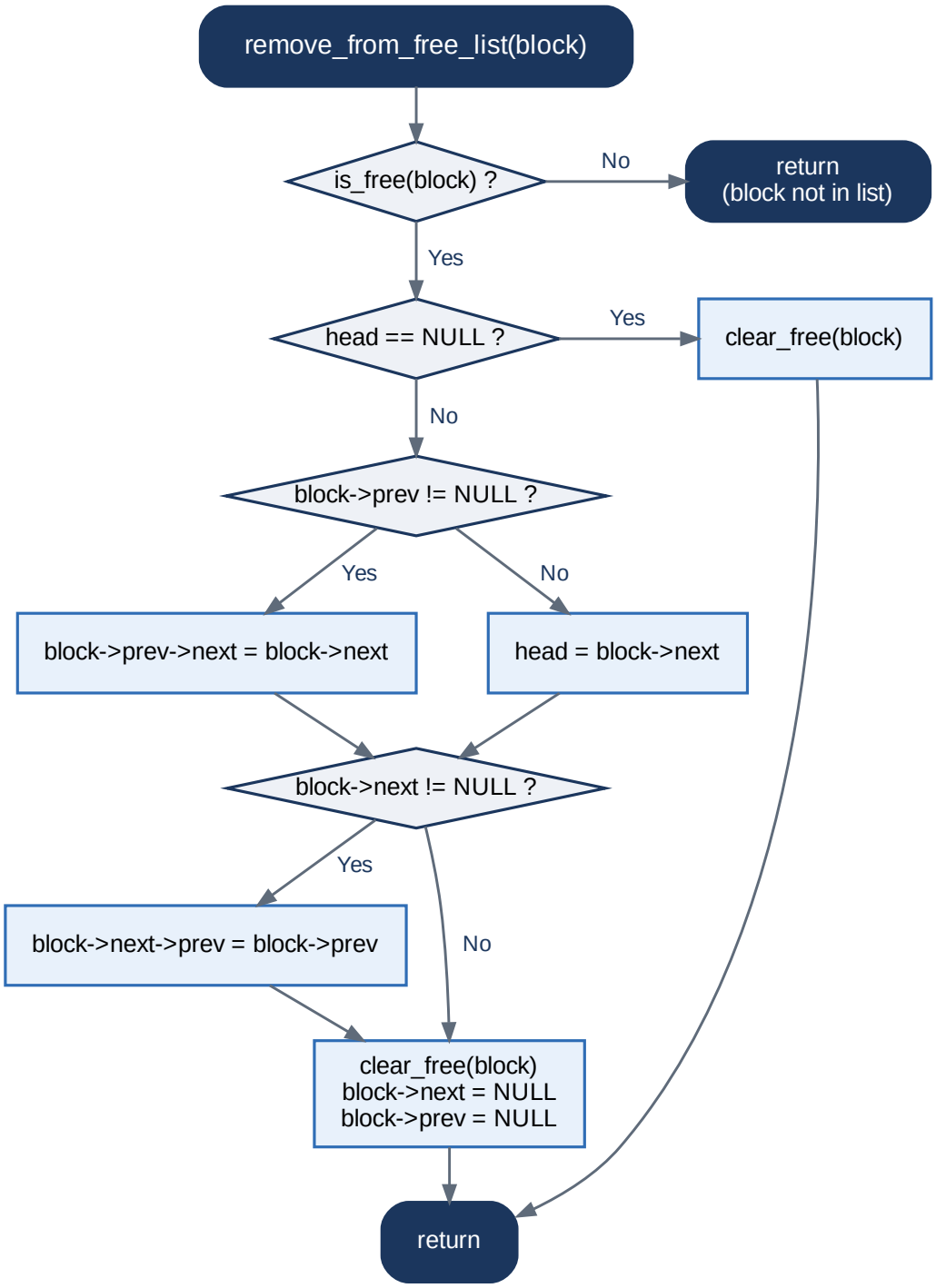


Fig. 5d

remove_from_free_list(block)

Unlinks a block from the doubly linked list; safe to call on a block that is not free (returns at once)



Free-list structure. Every free block stores `next` and `prev` inside its former payload, so the list costs no extra memory. The global `head` points to the smallest block. Because the list is sorted ascending, `find_block()` returns the best fit without scanning the whole list, and removal or insertion only touches the neighbours of the block involved.

Fig. 5e

expand_heap(size)

Asks the kernel for more memory with sbrk(); the very first call also aligns the program break

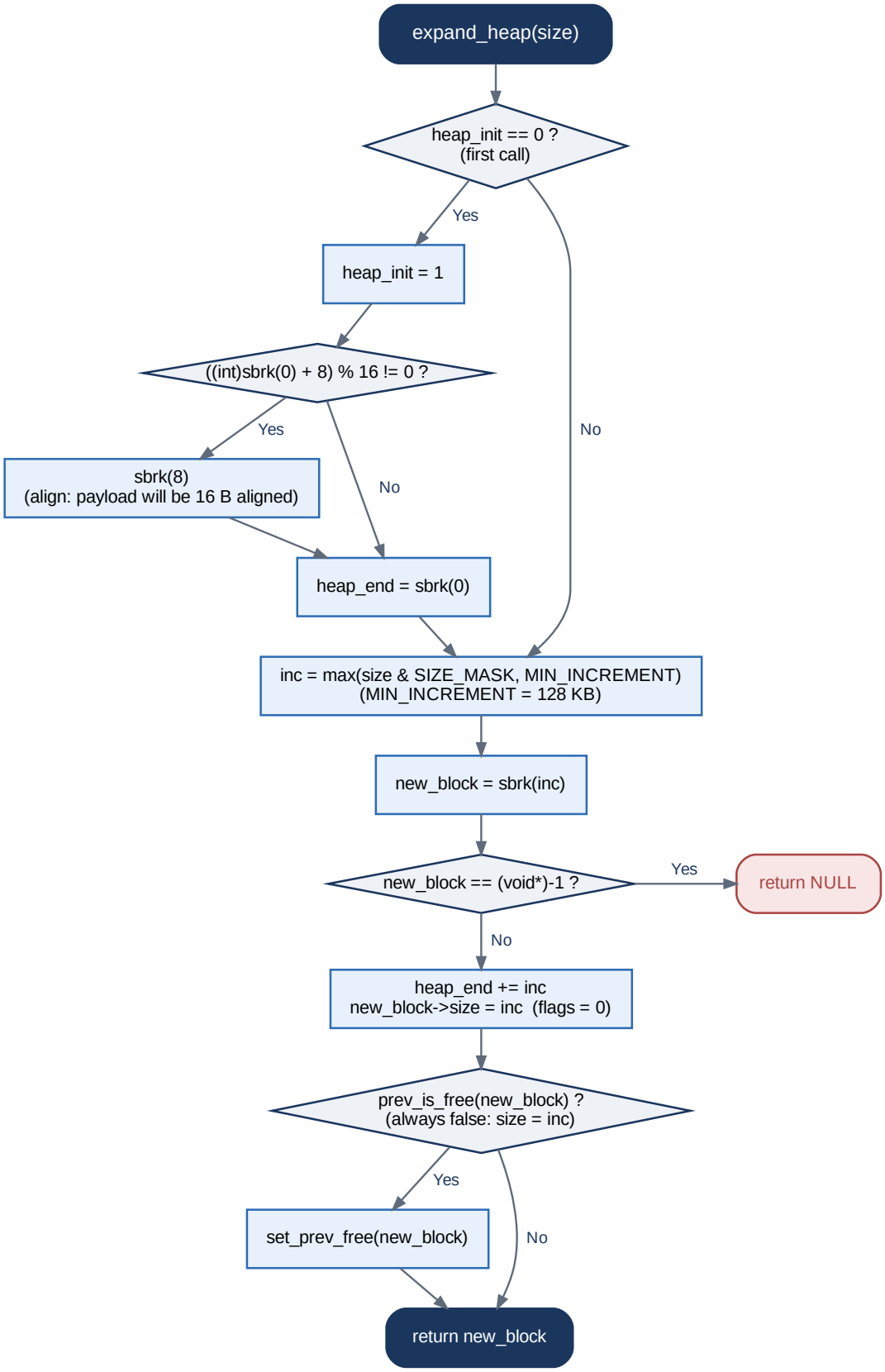
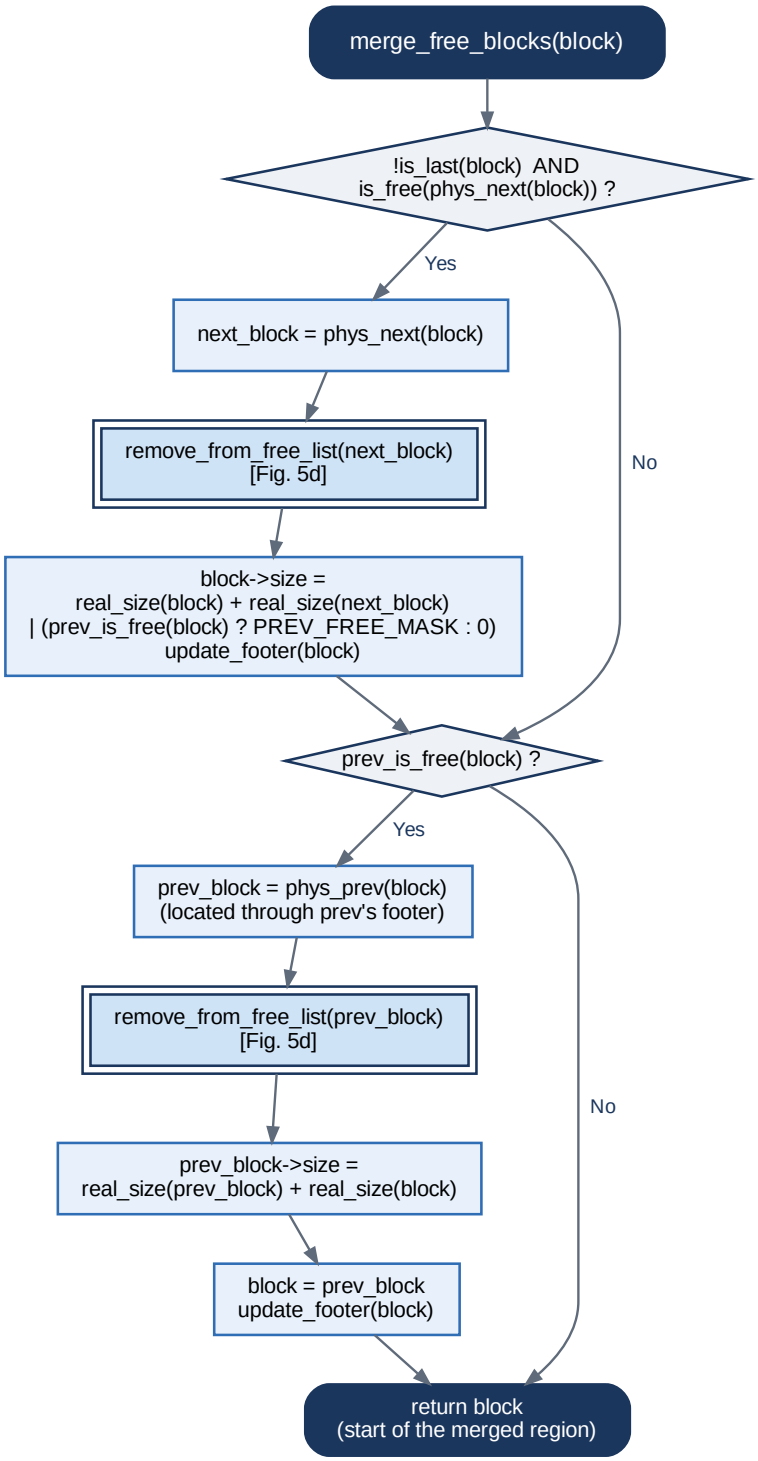


Fig. 5f

merge_free_blocks(block)

Forward merge first, then backward merge; returns the start of the merged region



Merge cases

P previous B freed block N next (dashed = free)

1 Neither neighbour free



no merge; B is simply filed in the free list

2 Next block free



B absorbs N (N leaves the list)

3 Previous block free



P absorbs B (P leaves the list, then re-enters with the new size)

4 Both neighbours free



P + B + N become one block starting at P

Note. The merged size is written back to the header, and `update_footer()` refreshes the footer so the block after it can still find its start. Merging never touches payload bytes.

Fig. 6a libc wrappers and hmmCalloc(nmemb, size)

heapWrappers.c exports the standard symbols so that linking or preloading libhmm replaces the system allocator

libc symbol	Forwards to	Notes
malloc(size)	HmmAlloc(size)	Fig. 3
free(ptr)	HmmFree(ptr)	Fig. 4
calloc(nmemb, size)	hmmCalloc(nmemb, size)	Fig. 6a; zero-fills the block
realloc(ptr, size)	hmmReAlloc(ptr, size)	Fig. 6b; may move the block

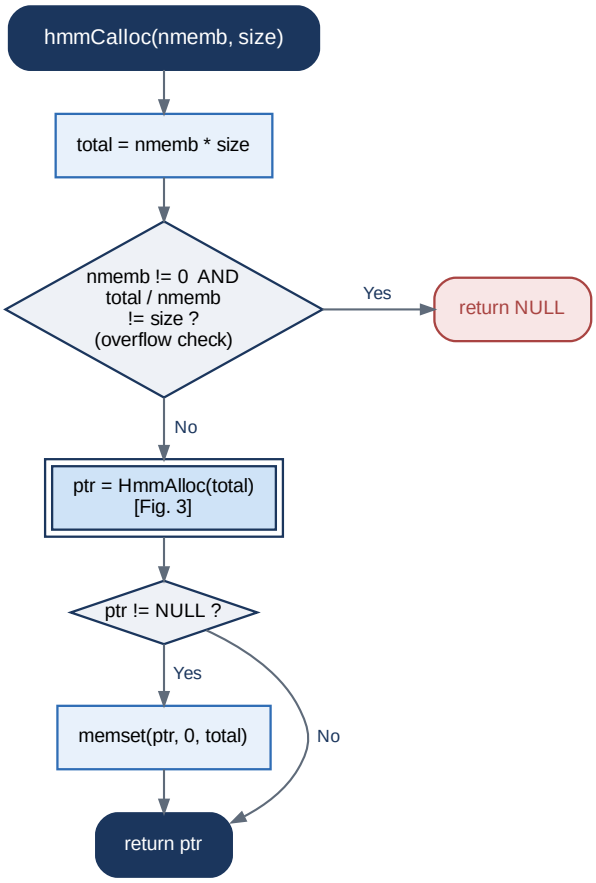
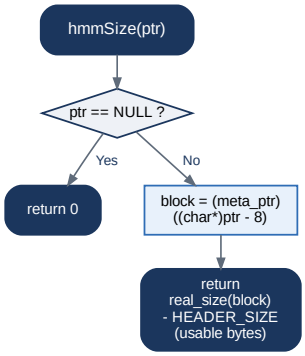


Fig. 6c hmmSize(ptr)



hmmSize reports the usable payload: block size minus the 8-byte header. The request `malloc(100)` gives a 112-byte block, so hmmSize returns 104.

hmmCalloc checks `nmemb * size` for overflow before allocating. After a successful allocation the whole requested size is cleared with memset.

Edge-case behaviour

Call	Result
malloc(0) / calloc(0, m) / calloc(n, 0)	NULL (size 0 is rejected before any work)
calloc(n, m) with n * m overflowing	NULL
realloc(NULL, n)	same as malloc(n)
realloc(p, 0)	frees p and returns NULL
realloc(p, n) with n <= usable size of p	returns p unchanged
realloc(p, n) when allocation fails	NULL, the original block stays valid
free(NULL) / free of a block already marked free	ignored

Fig. 6b

hmmReAlloc(ptr, reqSize)

Never shrinks and never grows in place: a larger request always allocates, copies and frees

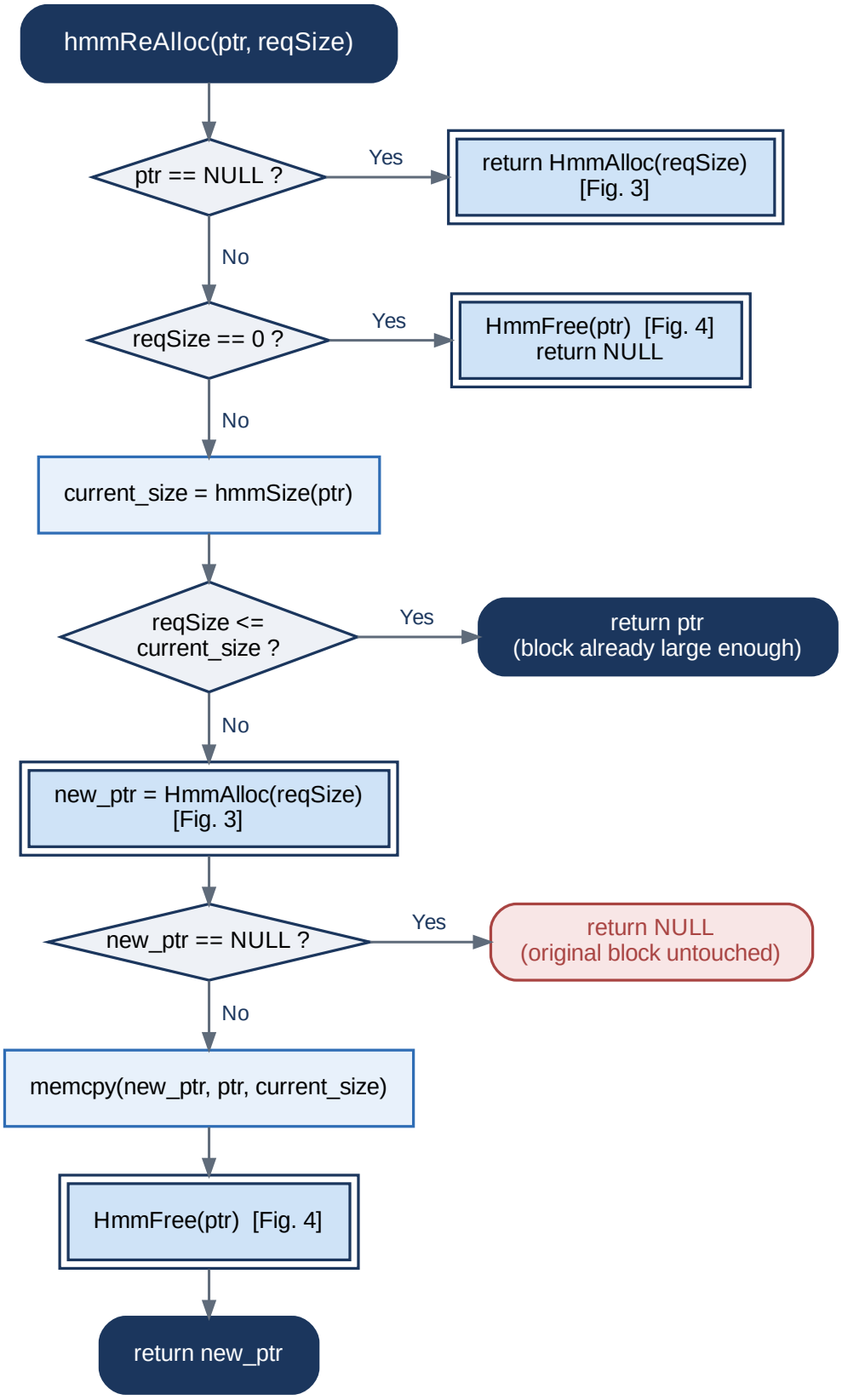


Fig. 8 Worked example: allocation and free sequence

Verified by running the code: A = malloc(100), B = malloc(50), C = malloc(200), then free(B), free(A), free(C)

allocated | free block | free remainder (not to scale) | PF PREV_FREE flag set

0 Start

The heap has not been created yet. head = NULL.

empty heap
no memory obtained from the kernel yet

```
free list: (empty)
```

1 A = malloc(100)

ALIGN(100 + 8) = 112. No free block fits, so expand_heap() calls sbrk(131072). The new 128 KiB block is split: 112 B for A, the rest returns to the list.

A 112	free remainder 130,960

```
free list: head -> [130960]
```

2 B = malloc(50)

ALIGN(58) = 64. find_block() picks the only free block, split_block() cuts 64 B off its front.

A 112	B 64	free remainder 130,896

```
free list: head -> [130896]
```

3 C = malloc(200)

ALIGN(208) = 208. Same path: best fit, split.

A 112	B 64	C 208	free remainder 130,688

```
free list: head -> [130688]
```

4 free(B)

Neighbours A and C are both in use, so no merge. B is inserted in size order and C gets the PREV_FREE flag.

A 112	B 64 free	C 208 PF	free remainder 130,688

```
free list: head -> [64] <-> [130688]
```

5 free(A)

The next block B is free: merge_free_blocks() absorbs it (B leaves the list), giving one free block of 176 B.

A + B 176 free	C 208 PF	free remainder 130,688

```
free list: head -> [176] <-> [130688]
```

6 free(C)

Forward merge with the free remainder, then backward merge with A + B: one block of 131,072 B reaches heap_end and is not below MIN_DECREMENT, so HmmFree() calls sbrk(-131072) instead of listing it.

A + B + C + remainder = 131,072 B (returned to the OS)

```
free list: (empty)   heap_end back at its starting address
```